

Day 107: K-Nearest Neighbors (KNN)

1. Day number

Day 107

2. Topic name

K-Nearest Neighbors Classification

3. Connection to yesterday

Yesterday, you learned **Logistic Regression**.

Logistic Regression learns from training data and creates a way to separate classes.

Today, you will learn **K-Nearest Neighbors (KNN)**.

KNN uses a different idea:

| Look at the examples closest to the new point and let those nearby examples vote.

So the basic contrast is:

Logistic Regression → learns a classification boundary KNN → looks at nearby training examples
--

4. Important topics

Neighbor

A **neighbor** is a training example that is close to the new example we want to classify.

Suppose customers are represented using:

Feature 1 = monthly visits Feature 2 = monthly purchases

Customers with similar values appear close together.

Distance idea

KNN needs some way to decide which points are closest.

For now, simply think:

Small distance → points are similar/close Large distance → points are farther apart
--

You do not need advanced distance formulas yet.

k

The letter **k** tells KNN how many nearby examples to consider.

For example:

```
k = 1
```

means:

| Look at only the nearest neighbor.

And:

```
k = 3
```

means:

| Look at the three nearest neighbors.

Majority vote

Once KNN finds the nearest k points, their classes vote.

For example, with k = 3:

```
Neighbor 1 → Class A  
Neighbor 2 → Class B  
Neighbor 3 → Class A
```

Votes:

```
Class A = 2  
Class B = 1
```

Prediction:

```
Class A
```

Classification

Because the final answer is a category such as Class A or Class B, KNN can be used for **classification**.

5. Foundational notes

Unlike many machine-learning algorithms, KNN does not learn a complicated equation during training.

Instead, it mainly keeps the training examples available.

When a new point arrives, KNN roughly does this:

```
New point
↓
Measure closeness to training points
↓
Find nearest k points
↓
Count their classes
↓
Choose majority class
```

This makes KNN very intuitive for beginners.

One important idea is that the meaning of **near** depends on your features.

For example, if your features are:

```
height
weight
```

two people with similar height and weight might appear close together.

6. Visual explanation and simple analogy

Imagine moving to a new neighborhood and trying to guess whether the area is mostly:

```
 Residential
or
 Commercial
```

Instead of studying the entire city, you look at the buildings immediately around you.

Suppose your three closest buildings are:

```
 Residential
 Residential
 Commercial
```

The majority is residential.

So you predict:

```
Residential
```

KNN works similarly.

For $k = 3$, it asks:

| What classes do my three closest training examples belong to?

You can experiment with that idea here by moving the new point or changing k :

Notice that changing k can sometimes change the predicted class.

7. Easy example

Suppose we have two features:

Feature 1 = hours studied
Feature 2 = practice questions completed

And two classes:

0 = Needs Improvement
1 = Ready

Training data:

Hours studied	Practice questions	Class
1	2	0
2	2	0
2	3	0
6	7	1
7	8	1
8	7	1

Now imagine a new student:

Hours studied = 7
Practice questions = 7

That point is close to:

(6, 7) → Class 1
(7, 8) → Class 1
(8, 7) → Class 1

If:

$k = 3$

then the vote is:

Class 1 = 3 votes
Class 0 = 0 votes

So KNN would classify the new student as:

Class 1 → Ready

8. Problem statement

Consider this tiny dataset:

Feature X	Feature Y	Class
1	1	A
2	1	A
2	2	A
6	6	B
7	6	B
7	7	B

You receive a new point:

(6.5, 6.5)

Use:

$k = 3$

Your task is to explain how KNN would classify this point.

Work conceptually first:

1. Compare the new point with the existing points.
2. Identify approximately which three are closest.
3. Look at their classes.
4. Count the votes.
5. Choose the majority class.

After understanding the reasoning, reproduce the workflow with scikit-learn.

9. Concepts used

Features

The input coordinates:

X
Y

Each training example has two feature values.

Label

Each example belongs to:

Class A
or
Class B

The class is the target we want to predict.

Distance

KNN compares the new point with existing points to determine which ones are closest.

Neighbors

The closest training examples become the relevant neighbors.

k

k controls how many neighbors participate in voting.

Majority vote

The most common class among the selected neighbors becomes the prediction.

10. Thought process

Suppose the new point is:

(6.5, 6.5)

First, look at the training data.

Class A points are around:

(1, 1)
(2, 1)
(2, 2)

Class B points are around:

(6, 6)
(7, 6)
(7, 7)

Ask:

Which group is the new point closest to?

The new point:

(6.5, 6.5)

is clearly near the Class B points.

With:

$k = 3$

its nearest neighbors will likely be the three Class B examples.

Votes:

$A \rightarrow 0$

$B \rightarrow 3$

Therefore:

Predicted class $\rightarrow B$

The important reasoning is not the exact arithmetic yet. It is the workflow:

```
New point
↓
Find closest points
↓
Take k neighbors
↓
Count labels
↓
Majority wins
```

11. Beginner-friendly pseudocode

```
START

create training points

store the class for every training point

choose k = 3

receive a new point

find how far the new point is
from every training point

sort points from nearest to farthest

select the first 3 points

count their classes

if Class A has more votes:
    predict Class A
else if Class B has more votes:
    predict Class B

display prediction

END
```

Scikit-learn performs the neighbor-finding work for you.

12. Suggested solving approach — scikit-learn after conceptual reasoning

First solve the tiny example manually.

Make sure you can answer:

What is the new point?

What is k?

Which examples appear closest?

What classes do they have?

Which class gets the majority vote?

Then move to scikit-learn.

The relevant model is:

```
from sklearn.neighbors import KNeighborsClassifier
```

The overall workflow will resemble:

```
prepare X
prepare y

create KNeighborsClassifier with k = ?

fit model using X and y

create new point

predict its class
```

Conceptually:

```
model = KNeighborsClassifier(n_neighbors=?)

model.fit(?, ?)

prediction = model.predict(?)
```

Fill in the missing pieces yourself rather than copying a complete solution.

For one new two-feature point, remember that scikit-learn expects something shaped conceptually like:

```
[[feature_1, feature_2]]
```

13. Easy edge cases

Tie

Suppose:

$k = 4$

and the nearest neighbors contain:

A
A
B
B

The vote is:

A = 2
B = 2

Now there is no simple majority.

Libraries have rules for handling situations like this, but as a beginner, the main lesson is:

| Some choices of k can produce ties.

For two-class problems, using an **odd** k , such as 3 or 5, can reduce the chance of a voting tie.

Very small k

Suppose:

$k = 1$

Only the single closest point matters.

That can make KNN very sensitive to one strange or noisy example.

For instance:

Nearby points:

A
A
A

but one unusually close noisy point:

B

With $k = 1$, the prediction might become:

B

With $k = 3$, surrounding points may have more influence.

This does **not** mean a particular k is always best. It simply shows why k matters.

14. Expected classification

For the exercise:

```
Class A:
(1,1)
(2,1)
(2,2)

Class B:
(6,6)
(7,6)
(7,7)

New point:
(6.5,6.5)

k = 3
```

The three nearest examples are around the Class B cluster.

So the expected vote is approximately:

```
B → 3 votes
A → 0 votes
```

Therefore:

```
Expected classification: Class B
```

15. Hint only

Think of the exercise like this:

```
training points + labels
    ↓
new point = [?, ?]
    ↓
choose k = ?
    ↓
find nearest neighbors
    ↓
look at their labels
    ↓
majority vote
    ↓
predicted class
```

For scikit-learn, the key pieces you need to investigate are:

```
KNeighborsClassifier(n_neighbors=...)
```

followed conceptually by:

```
fit(...)
predict(...)
```

The key idea from **Day 107** is:

KNN classifies a new example by looking at the k closest known examples and using their majority class as the prediction.